

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«Процедуры, функции, триггеры в PostgreSQL»

Обучающийся: Лютый Никита Артемович

Факультет: Прикладной информатики

Группа: К3240

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург
2026

Содержание

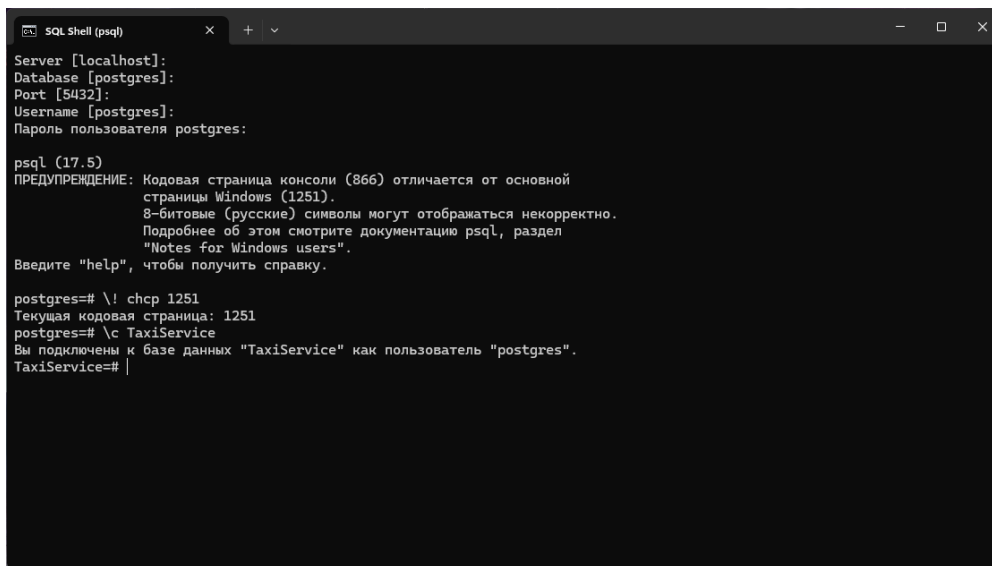
1	Цель работы	2
2	Работа в консольном клиенте SQL Shell (psql)	2
3	Хранимые процедуры и функции	2
3.1	Вывод пассажиров в заданном временном интервале	3
3.2	Поиск адресов доставки по номеру телефона	4
3.3	Вычисление дохода за месяц (параметр INOUT)	5
4	Реализация триггерных механизмов (10 триггеров)	6
4.1	Авторасчет стоимости	6
4.2	Защита от скрутки пробега	7
4.3	Форматирование телефона	8
4.4	Запрет онлайн-оплаты без карты в базе	9
4.5	Обновление даты ТО при достижении пробега	10
4.6	Логирование действий с персоналом	11
4.7	Автоматическое обновление пробега	12
4.8	Запрет удаления тарифов с историей	13
4.9	Проверка квалификации водителя	14
4.10	Обновление зарплаты	15
5	Ответы на контрольные вопросы	16
6	Выводы	16

1 Цель работы

Овладение практическими навыками разработки серверной логики на языке PL/pgSQL: создание хранимых процедур для аналитики и триггеров для обеспечения целостности данных и автоматизации бизнес-процессов таксопарка.

2 Работа в консольном клиенте SQL Shell (psql)

Согласно методическим указаниям, работа проводилась через консольный интерфейс. Для корректного отображения кириллицы использовалась команда `\!chcp 1251`.



```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Пароль пользователя postgres:

psql (17.5)
ПРЕДУПРЕЖДЕНИЕ: Кодовая страница консоли (866) отличается от основной
страницы Windows (1251).
8-битовые (русские) символы могут отображаться некорректно.
Подробнее об этом смотрите документацию psql, раздел
"Notes for Windows users".
Введите "help", чтобы получить справку.

postgres=# \! chcp 1251
Текущая кодовая страница: 1251
postgres=# \c TaxiService
Вы подключены к базе данных "TaxiService" как пользователь "postgres".
TaxiService=#
```

Рис. 1: Подключение к базе данных TaxiService через psql

3 Хранимые процедуры и функции

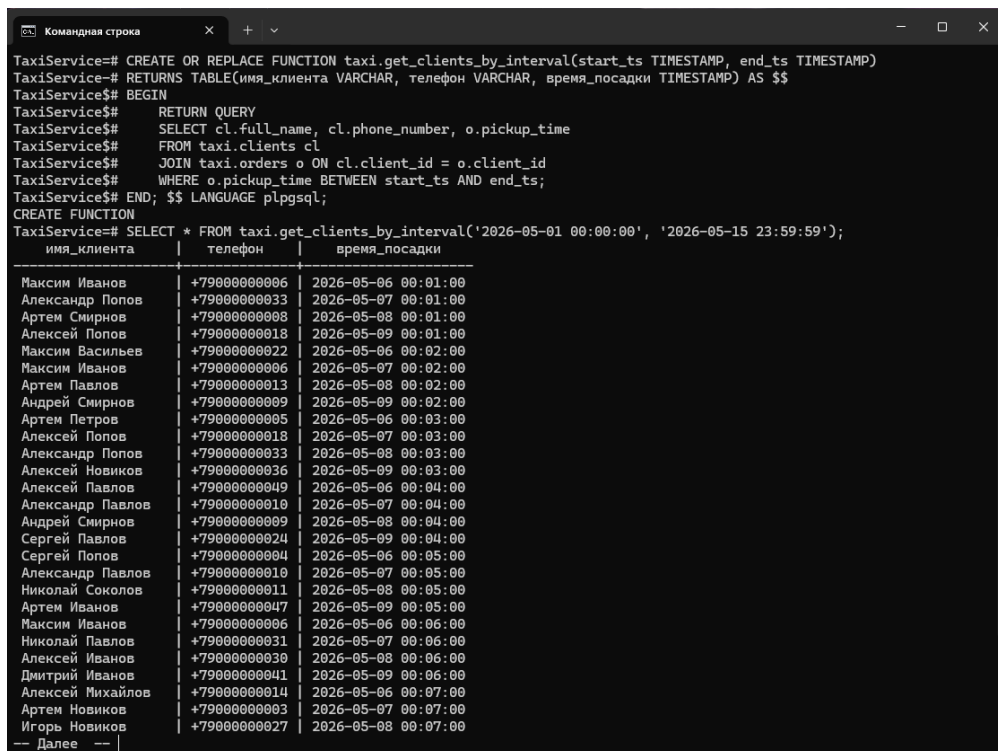
Были разработаны три объекта для автоматизации работы с пассажирами и финансами.

3.1 Вывод пассажиров в заданном временном интервале

```
CREATE OR REPLACE FUNCTION taxi.get_clients_by_interval(start_ts
    TIMESTAMP, end_ts TIMESTAMP)
RETURNS TABLE(name VARCHAR, phone VARCHAR, pickup_time TIMESTAMP) AS $$
BEGIN
    RETURN QUERY
    SELECT cl.full_name, cl.phone_number, o.pickup_time
    FROM taxi.clients cl
    JOIN taxi.orders o ON cl.client_id = o.client_id
    WHERE o.pickup_time BETWEEN start_ts AND end_ts;
END; $$ LANGUAGE plpgsql;

-- Проверка:
SELECT * FROM taxi.get_clients_by_interval('2026-05-01 00:00:00', '
    2026-05-15 23:59:59');
```

Листинг 1: Листинг вывода пассажиров в заданном временном интервале



имя_клиента	телефон	время_посадки
Максим Иванов	+79000000006	2026-05-06 00:01:00
Александр Попов	+79000000033	2026-05-07 00:01:00
Артем Смирнов	+79000000008	2026-05-08 00:01:00
Алексей Попов	+79000000018	2026-05-09 00:01:00
Максим Васильев	+79000000022	2026-05-06 00:02:00
Максим Иванов	+79000000006	2026-05-07 00:02:00
Артем Павлов	+79000000013	2026-05-08 00:02:00
Андрей Смирнов	+79000000009	2026-05-09 00:02:00
Артем Петров	+79000000005	2026-05-06 00:03:00
Алексей Попов	+79000000018	2026-05-07 00:03:00
Александр Попов	+79000000033	2026-05-08 00:03:00
Алексей Новиков	+79000000036	2026-05-09 00:03:00
Алексей Павлов	+79000000049	2026-05-06 00:04:00
Александр Павлов	+79000000010	2026-05-07 00:04:00
Андрей Смирнов	+79000000009	2026-05-08 00:04:00
Сергей Павлов	+79000000024	2026-05-09 00:04:00
Сергей Попов	+79000000004	2026-05-06 00:05:00
Александр Павлов	+79000000010	2026-05-07 00:05:00
Николай Соколов	+79000000011	2026-05-08 00:05:00
Артем Иванов	+79000000047	2026-05-09 00:05:00
Максим Иванов	+79000000006	2026-05-06 00:06:00
Николай Павлов	+79000000031	2026-05-07 00:06:00
Алексей Иванов	+79000000030	2026-05-08 00:06:00
Дмитрий Иванов	+79000000041	2026-05-09 00:06:00
Алексей Михайлов	+79000000014	2026-05-06 00:07:00
Артем Новиков	+79000000003	2026-05-07 00:07:00
Игорь Новиков	+79000000027	2026-05-08 00:07:00

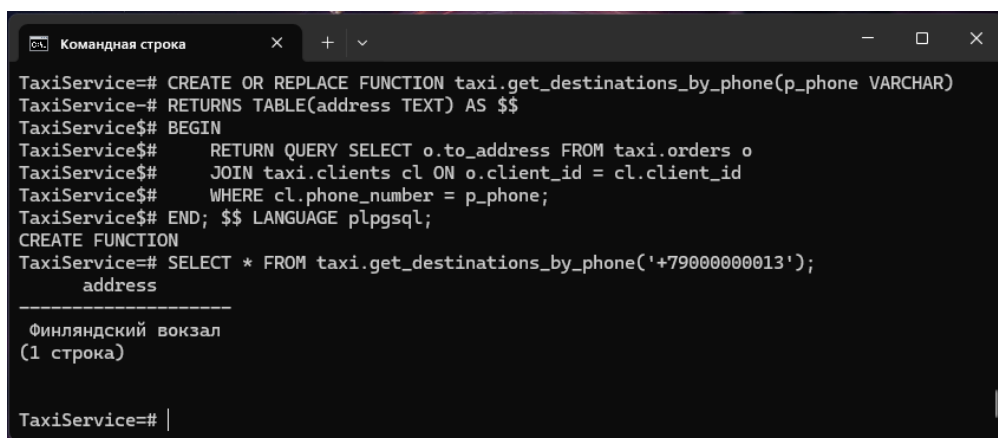
Рис. 2: Демонстрация вызова функции вывода пассажиров в заданном временном интервале

3.2 Поиск адресов доставки по номеру телефона

```
CREATE OR REPLACE FUNCTION taxi.get_destinations_by_phone(p_phone
    VARCHAR)
RETURNS TABLE(address TEXT) AS $$
BEGIN
    RETURN QUERY SELECT o.to_address FROM taxi.orders o
    JOIN taxi.clients cl ON o.client_id = cl.client_id
    WHERE cl.phone_number = p_phone;
END; $$ LANGUAGE plpgsql;

-- Проверка:
SELECT * FROM taxi.get_destinations_by_phone('+79000000013');
```

Листинг 2: Листинг поиска адресов доставки по номеру телефона



```
Командная строка
TaxiService=# CREATE OR REPLACE FUNCTION taxi.get_destinations_by_phone(p_phone VARCHAR)
TaxiService=# RETURNS TABLE(address TEXT) AS $$
TaxiService$# BEGIN
TaxiService$#     RETURN QUERY SELECT o.to_address FROM taxi.orders o
TaxiService$#     JOIN taxi.clients cl ON o.client_id = cl.client_id
TaxiService$#     WHERE cl.phone_number = p_phone;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# SELECT * FROM taxi.get_destinations_by_phone('+79000000013');
          address
-----
Финляндский вокзал
(1 строка)

TaxiService=# |
```

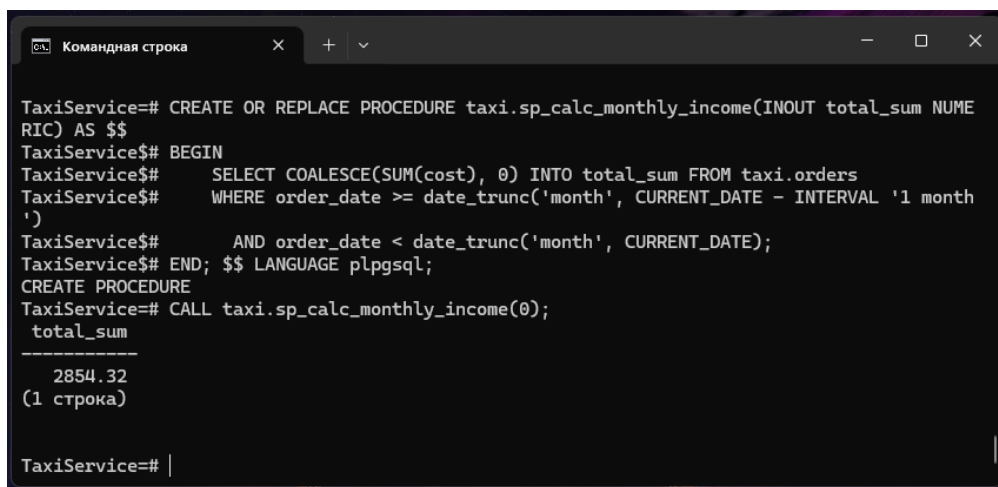
Рис. 3: Демонстрация вызова функции поиска адресов доставки по номеру телефона

3.3 Вычисление дохода за месяц (параметр INOUT)

```
CREATE OR REPLACE PROCEDURE taxi.sp_calc_monthly_income(INOUT total_sum
    NUMERIC) AS $$
BEGIN
    SELECT COALESCE(SUM(cost), 0) INTO total_sum FROM taxi.orders
    WHERE order_date >= date_trunc('month', CURRENT_DATE - INTERVAL '1
month')
    AND order_date < date_trunc('month', CURRENT_DATE);
END; $$ LANGUAGE plpgsql;

-- Проверка:
CALL taxi.sp_calc_revenue(0);
```

Листинг 3: Листинг вычисления дохода за месяц



The screenshot shows a terminal window titled "Командная строка" (Command Prompt). It displays the execution of a PL/SQL procedure. The user enters the following commands:

```
TaxiService=# CREATE OR REPLACE PROCEDURE taxi.sp_calc_monthly_income(INOUT total_sum NUMERIC) AS $$
TaxiService$# BEGIN
TaxiService$#     SELECT COALESCE(SUM(cost), 0) INTO total_sum FROM taxi.orders
TaxiService$#     WHERE order_date >= date_trunc('month', CURRENT_DATE - INTERVAL '1 month')
TaxiService$#     AND order_date < date_trunc('month', CURRENT_DATE);
TaxiService$# END; $$ LANGUAGE plpgsql;
TaxiService=# CALL taxi.sp_calc_monthly_income(0);
total_sum
```

The output shows the result of the procedure call:

```
-----
2854.32
(1 строка)
```

The prompt returns to "TaxiService=# |".

Рис. 4: Демонстрация вызова хранимой процедуры расчета дохода

4 Реализация триггерных механизмов (10 триггеров)

Разработана система из 10 триггеров, покрывающих аспекты работы такси.

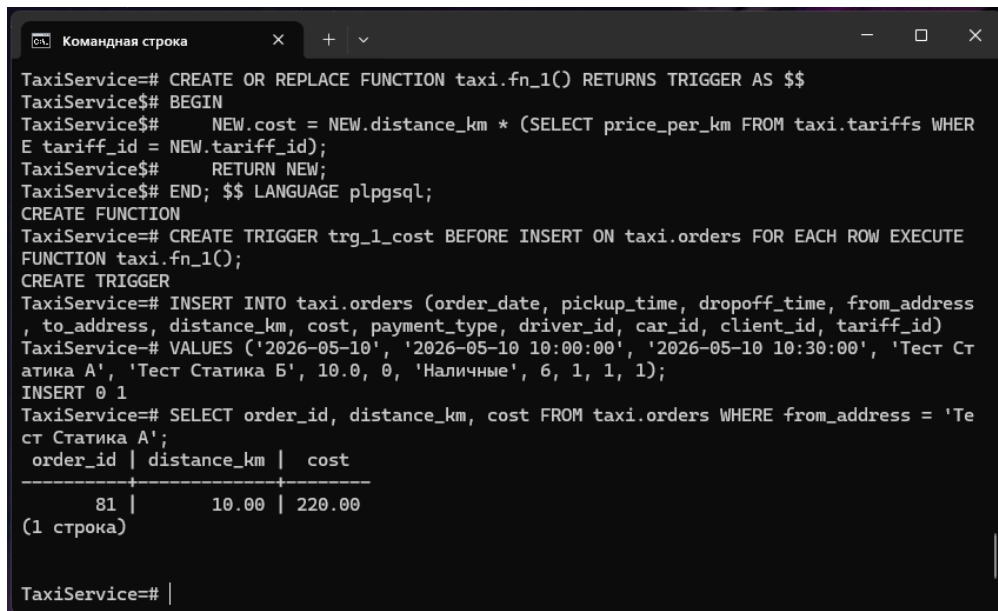
4.1 Авторасчет стоимости

```
CREATE OR REPLACE FUNCTION taxi.fn_1() RETURNS TRIGGER AS $$
BEGIN
    NEW.cost = NEW.distance_km * (SELECT price_per_km FROM taxi.tariffs
    WHERE tariff_id = NEW.tariff_id);
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_1_cost BEFORE INSERT ON taxi.orders FOR EACH ROW
EXECUTE FUNCTION taxi.fn_1();

-- Проверка:
INSERT INTO taxi.orders (order_date, pickup_time, dropoff_time,
    from_address, to_address, distance_km, cost, payment_type, driver_id
    , car_id, client_id, tariff_id)
VALUES ('2026-05-10', '2026-05-10 10:00:00', '2026-05-10 10:30:00', '
    Тест Статика А', 'Тест Статика Б', 10.0, 0, 'Наличные', 6, 1, 1, 1);

SELECT order_id, distance_km, cost FROM taxi.orders WHERE from_address
= 'Тест Статика А';
```

Листинг 4: Листинг триггера авторасчета стоимости (Дистанция * Тариф)



```
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_1() RETURNS TRIGGER AS $$
TaxiService$# BEGIN
TaxiService$#     NEW.cost = NEW.distance_km * (SELECT price_per_km FROM taxi.tariffs WHER
E tariff_id = NEW.tariff_id);
TaxiService$#     RETURN NEW;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_1_cost BEFORE INSERT ON taxi.orders FOR EACH ROW EXECUTE
FUNCTION taxi.fn_1();
CREATE TRIGGER
TaxiService=# INSERT INTO taxi.orders (order_date, pickup_time, dropoff_time, from_address
, to_address, distance_km, cost, payment_type, driver_id, car_id, client_id, tariff_id)
TaxiService=# VALUES ('2026-05-10', '2026-05-10 10:00:00', '2026-05-10 10:30:00', 'Тест Ст
атика А', 'Тест Статика Б', 10.0, 0, 'Наличные', 6, 1, 1, 1);
INSERT 0 1
TaxiService=# SELECT order_id, distance_km, cost FROM taxi.orders WHERE from_address = 'Те
ст Статика А';
 order_id | distance_km | cost
-----+-----+-----
      81 |      10.00 | 220.00
(1 строка)

TaxiService=#
```

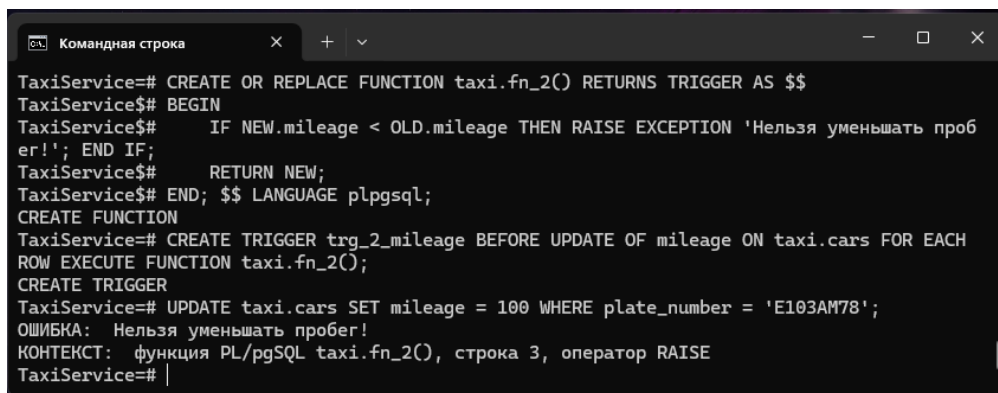
Рис. 5: Демонстрация работы триггера авторасчета стоимости

4.2 Защита от скрутки пробега

```
CREATE OR REPLACE FUNCTION taxi.fn_2() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.mileage < OLD.mileage THEN RAISE EXCEPTION 'Нельзя
    уменьшать пробег!'; END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_2_mileage BEFORE UPDATE OF mileage ON taxi.cars FOR
    EACH ROW EXECUTE FUNCTION taxi.fn_2();

-- Проверка:
UPDATE taxi.cars SET mileage = 100 WHERE plate_number = 'EAM10378';
```

Листинг 5: Листинг триггера защиты от скрутки пробега



The screenshot shows a terminal window titled "Командная строка" (Command Prompt). It contains the following SQL commands and their output:

```
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_2() RETURNS TRIGGER AS $$
TaxiService$# BEGIN
TaxiService$#     IF NEW.mileage < OLD.mileage THEN RAISE EXCEPTION 'Нельзя уменьшать пробег!'; END IF;
TaxiService$#     RETURN NEW;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_2_mileage BEFORE UPDATE OF mileage ON taxi.cars FOR EACH
ROW EXECUTE FUNCTION taxi.fn_2();
CREATE TRIGGER
TaxiService=# UPDATE taxi.cars SET mileage = 100 WHERE plate_number = 'E103AM78';
ОШИБКА: Нельзя уменьшать пробег!
КОНТЕКСТ: функция PL/pgSQL taxi.fn_2(), строка 3, оператор RAISE
TaxiService=#
```

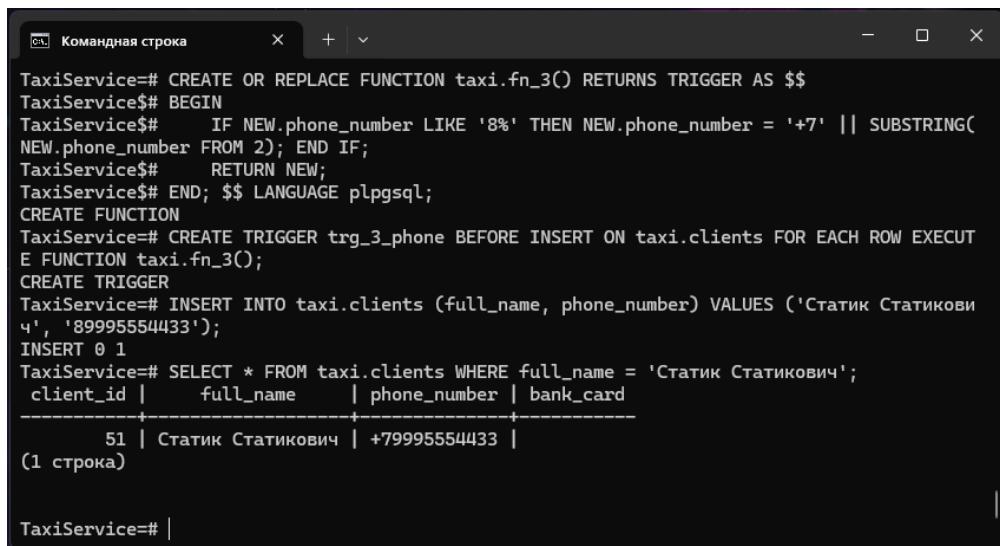
Рис. 6: Демонстрация работы триггера защиты от скрутки пробега

4.3 Форматирование телефона

```
CREATE OR REPLACE FUNCTION taxi.fn_3() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.phone_number LIKE '8%' THEN NEW.phone_number = '+7' ||
        SUBSTRING(NEW.phone_number FROM 2); END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_3_phone BEFORE INSERT ON taxi.clients FOR EACH ROW
EXECUTE FUNCTION taxi.fn_3();

-- Проверка:
INSERT INTO taxi.clients (full_name, phone_number) VALUES ('Статик
Статикович', '89995554433');
SELECT * FROM taxi.clients WHERE full_name = 'Статик Статикович';
```

Листинг 6: Листинг триггера форматирования телефона (8 -> +7)



The screenshot shows a terminal window titled "Командная строка" (Command Line). It displays the execution of the SQL code from Listing 6. The output shows the creation of the function and trigger, followed by an insert statement and a query that returns a single row from the taxi.clients table.

```
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_3() RETURNS TRIGGER AS $$
TaxiService$# BEGIN
TaxiService$#     IF NEW.phone_number LIKE '8%' THEN NEW.phone_number = '+7' || SUBSTRING(
NEW.phone_number FROM 2); END IF;
TaxiService$#     RETURN NEW;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_3_phone BEFORE INSERT ON taxi.clients FOR EACH ROW EXECUT
E FUNCTION taxi.fn_3();
CREATE TRIGGER
TaxiService=# INSERT INTO taxi.clients (full_name, phone_number) VALUES ('Статик Статикови
ч', '89995554433');
INSERT 0 1
TaxiService=# SELECT * FROM taxi.clients WHERE full_name = 'Статик Статикович';
 client_id |      full_name      | phone_number | bank_card
-----+-----+-----+-----
       51 | Статик Статикович | +79995554433 |
(1 строка)

TaxiService=#
```

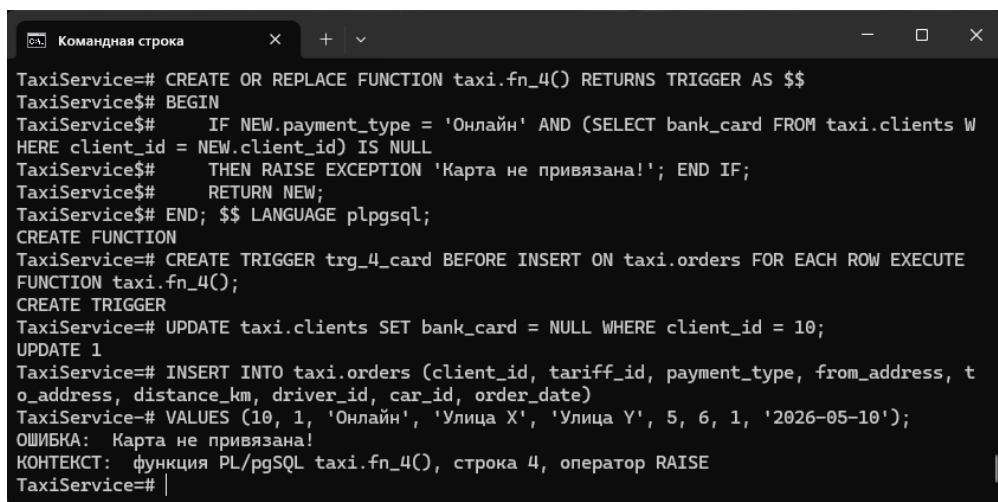
Рис. 7: Демонстрация работы триггера форматирования телефона

4.4 Запрет онлайн-оплаты без карты в базе

```
CREATE OR REPLACE FUNCTION taxi.fn_4() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.payment_type = 'Онлайн' AND (SELECT bank_card FROM taxi.
clients WHERE client_id = NEW.client_id) IS NULL
    THEN RAISE EXCEPTION 'Карта не привязана!'; END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_4_card BEFORE INSERT ON taxi.orders FOR EACH ROW
EXECUTE FUNCTION taxi.fn_4();

-- Проверка:
UPDATE taxi.clients SET bank_card = NULL WHERE client_id = 10;
INSERT INTO taxi.orders (client_id, tariff_id, payment_type,
    from_address, to_address, distance_km, driver_id, car_id, order_date
)
VALUES (10, 1, 'Онлайн', 'Улица X', 'Улица Y', 5, 6, 1, '2026-05-10');
```

Листинг 7: Листинг триггера запрета онлайн-оплаты без карты в базе



```
Командная строка
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_4() RETURNS TRIGGER AS $$
TaxiService$# BEGIN
TaxiService$#     IF NEW.payment_type = 'Онлайн' AND (SELECT bank_card FROM taxi.clients W
HERE client_id = NEW.client_id) IS NULL
TaxiService$#     THEN RAISE EXCEPTION 'Карта не привязана!'; END IF;
TaxiService$#     RETURN NEW;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_4_card BEFORE INSERT ON taxi.orders FOR EACH ROW EXECUTE
FUNCTION taxi.fn_4();
CREATE TRIGGER
TaxiService=# UPDATE taxi.clients SET bank_card = NULL WHERE client_id = 10;
UPDATE 1
TaxiService=# INSERT INTO taxi.orders (client_id, tariff_id, payment_type, from_address, t
o_address, distance_km, driver_id, car_id, order_date)
TaxiService=# VALUES (10, 1, 'Онлайн', 'Улица X', 'Улица Y', 5, 6, 1, '2026-05-10');
ОШИБКА:  Карта не привязана!
КОНТЕКСТ:  функция PL/pgSQL taxi.fn_4(), строка 4, оператор RAISE
TaxiService=#
```

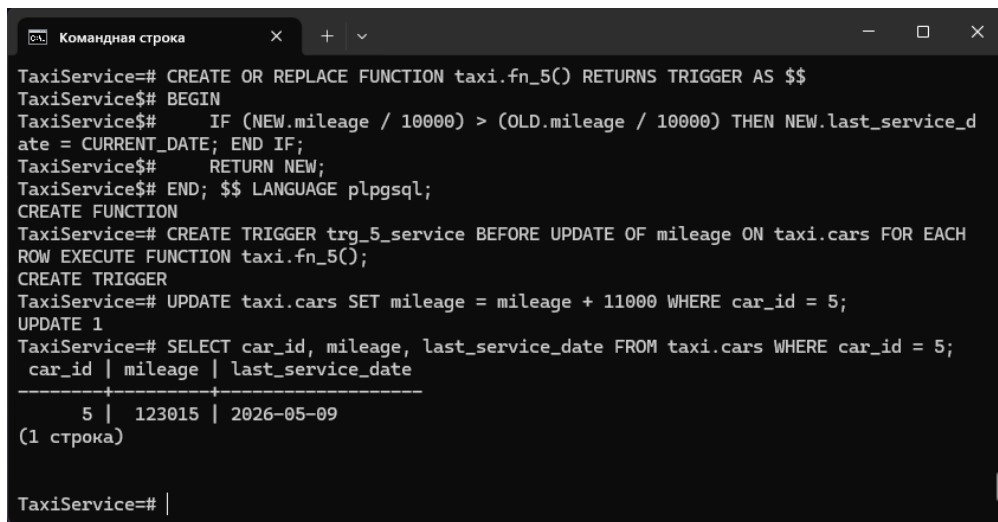
Рис. 8: Демонстрация работы триггера запрета онлайн-оплаты без карты в базе

4.5 Обновление даты ТО при достижении пробега

```
CREATE OR REPLACE FUNCTION taxi.fn_5() RETURNS TRIGGER AS $$
BEGIN
    IF (NEW.mileage / 10000) > (OLD.mileage / 10000) THEN NEW.
last_service_date = CURRENT_DATE; END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_5_service BEFORE UPDATE OF mileage ON taxi.cars FOR
EACH ROW EXECUTE FUNCTION taxi.fn_5();

-- Проверка:
UPDATE taxi.cars SET mileage = mileage + 11000 WHERE car_id = 5;
SELECT car_id, mileage, last_service_date FROM taxi.cars WHERE car_id =
5;
```

Листинг 8: Листинг триггера обновления даты ТО при достижении пробега (каждые 10000)



```
Командная строка
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_5() RETURNS TRIGGER AS $$
TaxiService$# BEGIN
TaxiService$#     IF (NEW.mileage / 10000) > (OLD.mileage / 10000) THEN NEW.last_service_d
ate = CURRENT_DATE; END IF;
TaxiService$#     RETURN NEW;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_5_service BEFORE UPDATE OF mileage ON taxi.cars FOR EACH
ROW EXECUTE FUNCTION taxi.fn_5();
CREATE TRIGGER
TaxiService=# UPDATE taxi.cars SET mileage = mileage + 11000 WHERE car_id = 5;
UPDATE 1
TaxiService=# SELECT car_id, mileage, last_service_date FROM taxi.cars WHERE car_id = 5;
 car_id | mileage | last_service_date
-----+-----+-----
      5 | 123015 | 2026-05-09
(1 строка)

TaxiService=# |
```

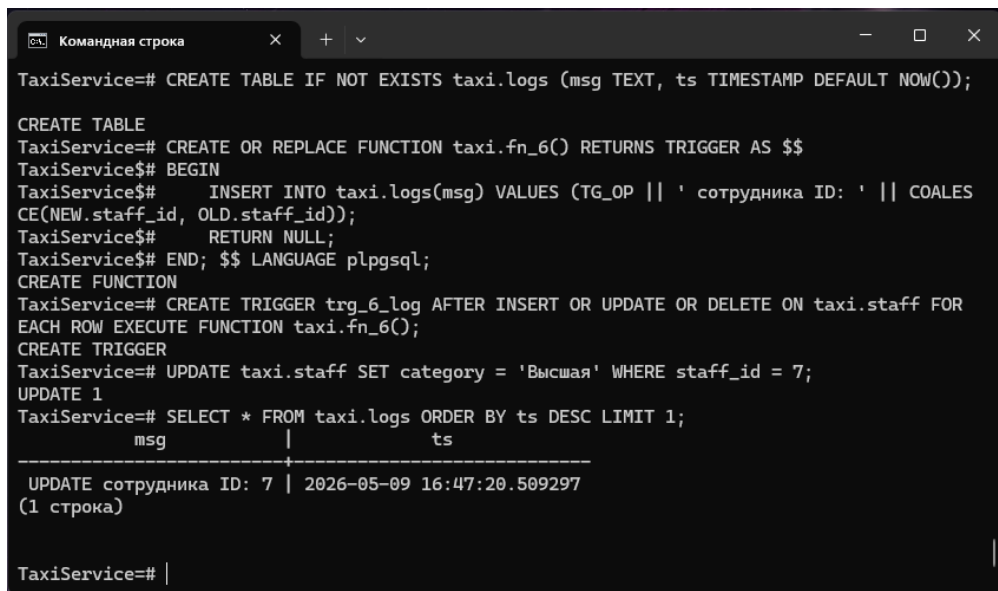
Рис. 9: Демонстрация работы триггера обновления даты ТО при достижении пробега

4.6 Логирование действий с персоналом

```
CREATE TABLE IF NOT EXISTS taxi.logs (msg TEXT, ts TIMESTAMP DEFAULT
NOW());
CREATE OR REPLACE FUNCTION taxi.fn_6() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO taxi.logs(msg) VALUES (TG_OP || ' сотрудника ID: ' ||
COALESCE(NEW.staff_id, OLD.staff_id));
    RETURN NULL;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_6_log AFTER INSERT OR UPDATE OR DELETE ON taxi.staff
FOR EACH ROW EXECUTE FUNCTION taxi.fn_6();

-- Проверка:
UPDATE taxi.staff SET category = 'Высшая' WHERE staff_id = 7;
SELECT * FROM taxi.logs ORDER BY ts DESC LIMIT 1;
```

Листинг 9: Листинг триггера логирования действий с персоналом



```
Командная строка
TaxiService=# CREATE TABLE IF NOT EXISTS taxi.logs (msg TEXT, ts TIMESTAMP DEFAULT NOW());
CREATE TABLE
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_6() RETURNS TRIGGER AS $$
TaxiService$# BEGIN
TaxiService$#     INSERT INTO taxi.logs(msg) VALUES (TG_OP || ' сотрудника ID: ' || COALES
CE(NEW.staff_id, OLD.staff_id));
TaxiService$#     RETURN NULL;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_6_log AFTER INSERT OR UPDATE OR DELETE ON taxi.staff FOR
EACH ROW EXECUTE FUNCTION taxi.fn_6();
CREATE TRIGGER
TaxiService=# UPDATE taxi.staff SET category = 'Высшая' WHERE staff_id = 7;
UPDATE 1
TaxiService=# SELECT * FROM taxi.logs ORDER BY ts DESC LIMIT 1;
      msg      |      ts
-----+-----
UPDATE сотрудника ID: 7 | 2026-05-09 16:47:20.509297
(1 строка)

TaxiService=#
```

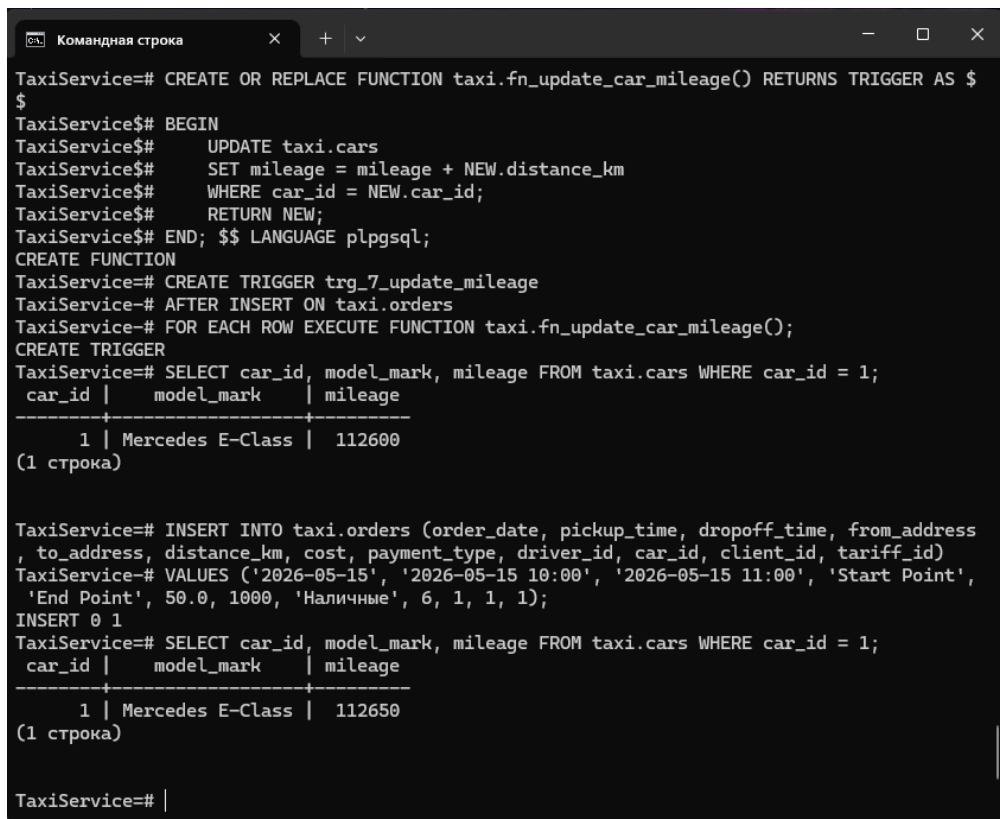
Рис. 10: Демонстрация работы триггера логирования действий с персоналом

4.7 Автоматическое обновление пробега

```
CREATE OR REPLACE FUNCTION taxi.fn_update_car_mileage() RETURNS TRIGGER
AS $$
BEGIN
    UPDATE taxi.cars
    SET mileage = mileage + NEW.distance_km
    WHERE car_id = NEW.car_id;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_7_update_mileage
AFTER INSERT ON taxi.orders
FOR EACH ROW EXECUTE FUNCTION taxi.fn_update_car_mileage();

-- Проверка:
SELECT car_id, model_mark, mileage FROM taxi.cars WHERE car_id = 1;
INSERT INTO taxi.orders (order_date, pickup_time, dropoff_time,
    from_address, to_address, distance_km, cost, payment_type, driver_id
    , car_id, client_id, tariff_id)
VALUES ('2026-05-15', '2026-05-15 10:00', '2026-05-15 11:00', 'Start
    Point', 'End Point', 50.0, 1000, 'Наличные', 6, 1, 1, 1);
SELECT car_id, model_mark, mileage FROM taxi.cars WHERE car_id = 1;
```

Листинг 10: Листинг триггера автоматического обновления пробега



```
Командная строка
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_update_car_mileage() RETURNS TRIGGER AS $
$
TaxiService$# BEGIN
TaxiService$#     UPDATE taxi.cars
TaxiService$#     SET mileage = mileage + NEW.distance_km
TaxiService$#     WHERE car_id = NEW.car_id;
TaxiService$#     RETURN NEW;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_7_update_mileage
TaxiService=# AFTER INSERT ON taxi.orders
TaxiService=# FOR EACH ROW EXECUTE FUNCTION taxi.fn_update_car_mileage();
CREATE TRIGGER
TaxiService=# SELECT car_id, model_mark, mileage FROM taxi.cars WHERE car_id = 1;
car_id | model_mark | mileage
-----+-----+-----
1 | Mercedes E-Class | 112600
(1 строка)

TaxiService=# INSERT INTO taxi.orders (order_date, pickup_time, dropoff_time, from_address
, to_address, distance_km, cost, payment_type, driver_id, car_id, client_id, tariff_id)
TaxiService=# VALUES ('2026-05-15', '2026-05-15 10:00', '2026-05-15 11:00', 'Start
'End Point', 50.0, 1000, 'Наличные', 6, 1, 1, 1);
INSERT 0 1
TaxiService=# SELECT car_id, model_mark, mileage FROM taxi.cars WHERE car_id = 1;
car_id | model_mark | mileage
-----+-----+-----
1 | Mercedes E-Class | 112650
(1 строка)

TaxiService=#
```

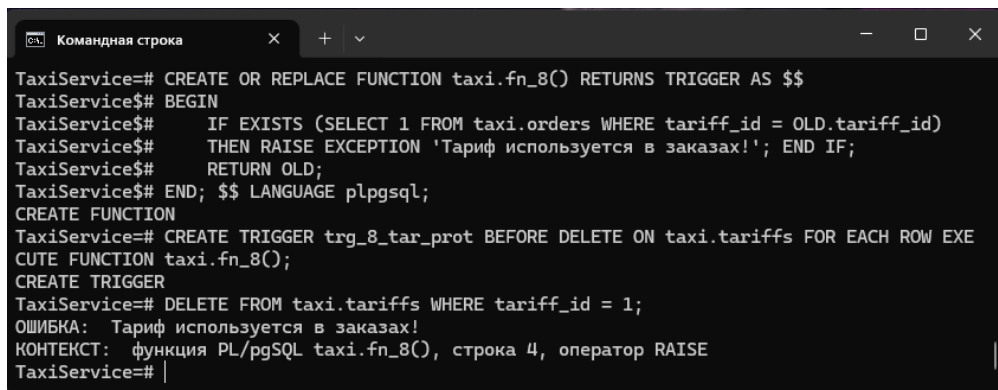
Рис. 11: Демонстрация работы триггера автоматического обновления пробега

4.8 Запрет удаления тарифов с историей

```
CREATE OR REPLACE FUNCTION taxi.fn_8() RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM taxi.orders WHERE tariff_id = OLD.
tariff_id)
        THEN RAISE EXCEPTION 'Тариф используется в заказах!'; END IF;
    RETURN OLD;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_8_tar_prot BEFORE DELETE ON taxi.tariffs FOR EACH
ROW EXECUTE FUNCTION taxi.fn_8();

-- Проверка:
DELETE FROM taxi.tariffs WHERE tariff_id = 1;
```

Листинг 11: Листинг триггера запрета удаления тарифов с историей



The screenshot shows a terminal window titled "Командная строка" (Command Prompt). It displays the execution of the SQL code from Listing 11. The commands are entered line by line, and the prompt "TaxiService=#" is visible. The error message "ОШИБКА: Тариф используется в заказах!" (Error: Tariff is used in orders!) is shown, along with the context "КОНТЕКСТ: функция PL/pgSQL taxi.fn_8(), строка 4, оператор RAISE" (Context: PL/pgSQL function taxi.fn_8(), line 4, operator RAISE).

```
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_8() RETURNS TRIGGER AS $$
TaxiService$# BEGIN
TaxiService$#     IF EXISTS (SELECT 1 FROM taxi.orders WHERE tariff_id = OLD.tariff_id)
TaxiService$#         THEN RAISE EXCEPTION 'Тариф используется в заказах!'; END IF;
TaxiService$#         RETURN OLD;
TaxiService$#     END; $$ LANGUAGE plpgsql;
TaxiService=# CREATE TRIGGER trg_8_tar_prot BEFORE DELETE ON taxi.tariffs FOR EACH ROW EXE
CUTE FUNCTION taxi.fn_8();
TaxiService=# CREATE TRIGGER
TaxiService=# DELETE FROM taxi.tariffs WHERE tariff_id = 1;
ОШИБКА: Тариф используется в заказах!
КОНТЕКСТ: функция PL/pgSQL taxi.fn_8(), строка 4, оператор RAISE
TaxiService=#
```

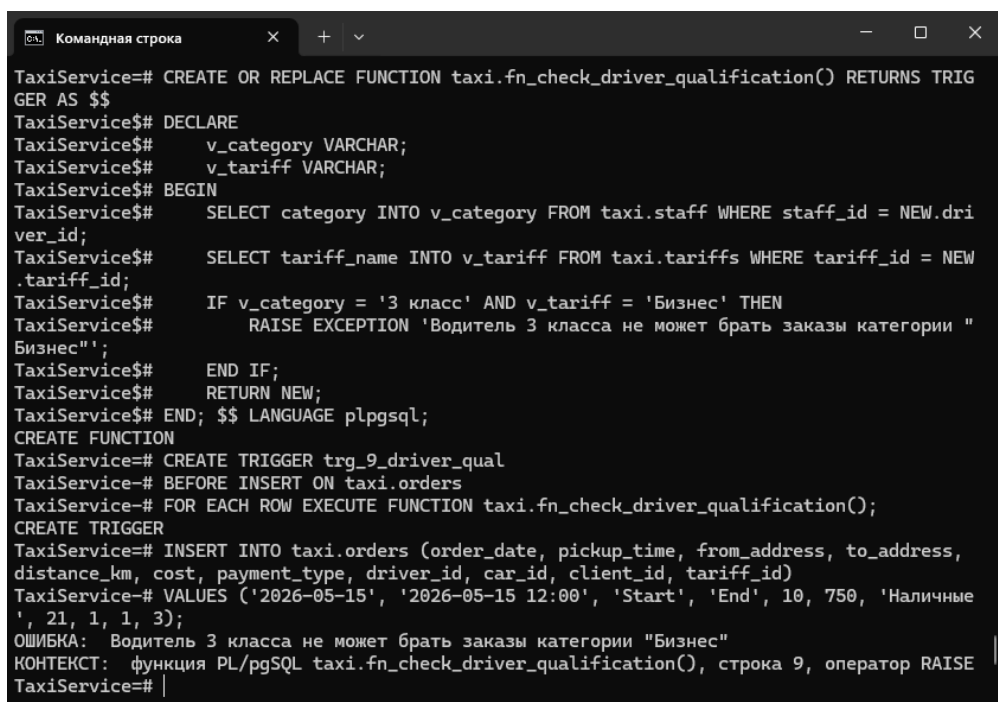
Рис. 12: Демонстрация работы триггера запрета удаления тарифов с историей

4.9 Проверка квалификации водителя

```
CREATE OR REPLACE FUNCTION taxi.fn_check_driver_qualification() RETURNS
    TRIGGER AS $$
DECLARE
    v_category VARCHAR;
    v_tariff VARCHAR;
BEGIN
    SELECT category INTO v_category FROM taxi.staff WHERE staff_id =
    NEW.driver_id;
    SELECT tariff_name INTO v_tariff FROM taxi.tariffs WHERE tariff_id
    = NEW.tariff_id;
    IF v_category = '3 класс' AND v_tariff = 'Бизнес' THEN
        RAISE EXCEPTION 'Водитель 3 класса не может брать заказы
    категории Бизнес';
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_9_driver_qual
BEFORE INSERT ON taxi.orders
FOR EACH ROW EXECUTE FUNCTION taxi.fn_check_driver_qualification();

-- Проверка:
INSERT INTO taxi.orders (order_date, pickup_time, from_address,
    to_address, distance_km, cost, payment_type, driver_id, car_id,
    client_id, tariff_id)
VALUES ('2026-05-15', '2026-05-15 12:00', 'Start', 'End', 10, 750, '
    Наличные', 21, 1, 1, 3);
```

Листинг 12: Листинг триггера проверки квалификации водителя



The screenshot shows a terminal window titled "Командная строка" (Command Prompt). It displays the execution of the SQL code from Listing 12. The commands are executed line by line, with the prompt changing from "TaxiService=#" to "TaxiService\$#" for declarations and "TaxiService\$#" for the trigger creation. The final command is the INSERT statement. The output shows the successful execution of the INSERT statement, followed by an error message: "ОШИБКА: Водитель 3 класса не может брать заказы категории "Бизнес"". The context provided is "контекст: функция PL/pgSQL taxi.fn_check_driver_qualification(), строка 9, оператор RAISE".

```
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_check_driver_qualification() RETURNS TRIG
GER AS $$
TaxiService$# DECLARE
TaxiService$#     v_category VARCHAR;
TaxiService$#     v_tariff VARCHAR;
TaxiService$# BEGIN
TaxiService$#     SELECT category INTO v_category FROM taxi.staff WHERE staff_id = NEW.dri
ver_id;
TaxiService$#     SELECT tariff_name INTO v_tariff FROM taxi.tariffs WHERE tariff_id = NEW
.tariff_id;
TaxiService$#     IF v_category = '3 класс' AND v_tariff = 'Бизнес' THEN
TaxiService$#         RAISE EXCEPTION 'Водитель 3 класса не может брать заказы категории "
Бизнес"';
TaxiService$#     END IF;
TaxiService$#     RETURN NEW;
TaxiService$# END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_9_driver_qual
TaxiService=# BEFORE INSERT ON taxi.orders
TaxiService=# FOR EACH ROW EXECUTE FUNCTION taxi.fn_check_driver_qualification();
CREATE TRIGGER
TaxiService=# INSERT INTO taxi.orders (order_date, pickup_time, from_address, to_address,
distance_km, cost, payment_type, driver_id, car_id, client_id, tariff_id)
TaxiService=# VALUES ('2026-05-15', '2026-05-15 12:00', 'Start', 'End', 10, 750, 'Наличные
', 21, 1, 1, 3);
ОШИБКА: Водитель 3 класса не может брать заказы категории "Бизнес"
КОНТЕКСТ: функция PL/pgSQL taxi.fn_check_driver_qualification(), строка 9, оператор RAISE
TaxiService=#
```

Рис. 13: Демонстрация работы триггера проверки квалификации водителя

4.10 Обновление зарплаты

```
CREATE OR REPLACE FUNCTION taxi.fn_auto_salary_accrual() RETURNS
    TRIGGER AS $$
DECLARE
    v_month_start DATE := date_trunc('month', CURRENT_DATE);
BEGIN
    IF EXISTS (SELECT 1 FROM taxi.salary WHERE staff_id = NEW.driver_id
        AND calc_date = v_month_start) THEN
        UPDATE taxi.salary
        SET total_amount = total_amount + (NEW.cost * 0.5)
        WHERE staff_id = NEW.driver_id AND calc_date = v_month_start;
    ELSE
        INSERT INTO taxi.salary (staff_id, calc_date, total_amount)
        VALUES (NEW.driver_id, v_month_start, NEW.cost * 0.5);
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_10_auto_salary
AFTER INSERT ON taxi.orders
FOR EACH ROW EXECUTE FUNCTION taxi.fn_auto_salary_accrual();

-- Проверка:
SELECT * FROM taxi.salary WHERE staff_id = 6;
INSERT INTO taxi.orders (order_date, pickup_time, from_address,
    to_address, distance_km, cost, payment_type, driver_id, car_id,
    client_id, tariff_id)
VALUES ('2026-05-15', '2026-05-15 13:00', 'Home', 'Work', 10, 1000, '
Наличные', 6, 1, 1, 1);
SELECT * FROM taxi.salary WHERE staff_id = 6;
```

Листинг 13: Листинг триггера обновления зарплаты


```
Командная строка
TaxiService=# CREATE OR REPLACE FUNCTION taxi.fn_auto_salary_accrual() RETURNS TRIGGER AS
$$
TaxiService## DECLARE
TaxiService##     v_month_start DATE := date_trunc('month', CURRENT_DATE);
TaxiService## BEGIN
TaxiService##     IF EXISTS (SELECT 1 FROM taxi.salary WHERE staff_id = NEW.driver_id AND
TaxiService##         calc_date = v_month_start) THEN
TaxiService##         UPDATE taxi.salary
TaxiService##         SET total_amount = total_amount + (NEW.cost * 0.5)
TaxiService##         WHERE staff_id = NEW.driver_id AND calc_date = v_month_start;
TaxiService##     ELSE
TaxiService##         INSERT INTO taxi.salary (staff_id, calc_date, total_amount)
TaxiService##         VALUES (NEW.driver_id, v_month_start, NEW.cost * 0.5);
TaxiService##     END IF;
TaxiService##     RETURN NEW;
TaxiService## END; $$ LANGUAGE plpgsql;
CREATE FUNCTION
TaxiService=# CREATE TRIGGER trg_10_auto_salary
TaxiService=# AFTER INSERT ON taxi.orders
TaxiService=# FOR EACH ROW EXECUTE FUNCTION taxi.fn_auto_salary_accrual();
CREATE TRIGGER
TaxiService=# SELECT * FROM taxi.salary WHERE staff_id = 6;
salary_id | staff_id | calc_date | total_amount
-----+-----+-----+-----
(0 строк)

TaxiService=# INSERT INTO taxi.orders (order_date, pickup_time, from_address, to_address,
distance_km, cost, payment_type, driver_id, car_id, client_id, tariff_id)
TaxiService=# VALUES ('2026-05-15', '2026-05-15 13:00', 'Home', 'Work', 10, 1000, 'Наличны
e', 6, 1, 1, 1);
INSERT 0 1
TaxiService=# SELECT * FROM taxi.salary WHERE staff_id = 6;
salary_id | staff_id | calc_date | total_amount
-----+-----+-----+-----
1 | 6 | 2026-05-01 | 110.00
(1 строка)

TaxiService=#
```

Рис. 14: Демонстрация работы триггера обновления зарплаты

5 Ответы на контрольные вопросы

1. В чем разница между процедурой и функцией?

Функция всегда возвращает значение (RETURNS) и может быть вложена в SELECT. Процедура вызывается через CALL, не возвращает результат напрямую, но поддерживает управление транзакциями (COMMIT/ROLLBACK).

2. Когда выполняются BEFORE и AFTER триггеры?

BEFORE выполняется до записи данных в таблицу (используется для валидации и изменения значений). AFTER выполняется после успешной операции (используется для логирования или каскадных изменений).

6 Выводы

В ходе лабораторной работы бизнес-логика службы такси была успешно перенесена на сторону СУБД. Это позволило автоматизировать расчеты, внедрить систему аудита персонала и обеспечить достоверность вводимых данных с помощью триггеров.